

Treasure Hunt

LEGAL NAME	<i>LEONARD-ADRIAN BUNEA</i>
PREFERRED NAME	ZOE MCFIFE
TIME SPENT	06:59:48

Table of Contents

Table of Contents	1
Table of Figures	2
Before we start	3
The Board	3
Generating the Maze	3
Displaying the Maze	5
Player Controls	7
Pathfinding	8
Greedy	10
MiniMax	11
Example Game State	15

Table of Figures

Figure 1: Maze Constructzor.....	3
Figure 2: Maze Carving.....	4
Figure 3: A Maze.....	5
Figure 4: printMaze	6
Figure 5: player movement.....	7
Figure 6: movePlayer	8
Figure 7: simple search algorithm	9
Figure 8: Inspirational Quote	10
Figure 9: Greedy	11
Figure 10: MiniMax.....	14
Figure 11: another maze! big!.....	15
Figure 12: Debug Output	16

Before we start

I want to apologize; the code is a bit messy this time! The Maze class has no right to be that large and can be split and optimized a lot... but oh well, it's good enough and I don't feel like cleaning it up.

The Board

A big empty board with just a few obstacles is boring! A maze is much more interesting.

Generating the Maze

I implemented a simple depth-first maze carving algorithm (thanks Wikipedia) to generate procedural mazes. The size of these can be set in the constructor. I created a Tile class to represent different tile types and represent grid positions using a TilePosition record.

```
public Maze(int size, int treasureCount, AiMode aiMode)
{
    this.size = size;
    this.aiMode = aiMode;
    grid = initializeMaze(size);

    spawnPlayer();
    spawnAI();
    spawnTreasures(treasureCount);
}
```

Figure 1: Maze Constructzor

```
private Tile[][] carveMaze(Tile[][] grid)
{
    carve(new TilePosition(0, 0), grid);
    return grid;
}

private void carve(TilePosition tile, Tile[][] grid)
```

Treasure Hunt

```
{
    if (!isValidTile(tile) || grid[tile.row()][tile.col()] !=
Tile.WALL)
    {
        return;
    }

    if (countAdjacentPaths(tile) > 1)
    {
        return;
    }

    grid[tile.row()][tile.col()] = Tile.PATH;

    List<TilePosition> neighbourTiles =
getDirectNeighbourTiles(tile);
    Collections.shuffle(neighbourTiles);
    (shuffling the neighbour tiles to make the maze, be a maze.
Would turn into straight lines otherwise)

    for (TilePosition neighbourTile : neighbourTiles)
    {
        if (getTileType(neighbourTile) == Tile.WALL)
        {
            carve(neighbourTile, grid);
        }
    }
}
```

Figure 2: Maze Carving

Treasure Hunt

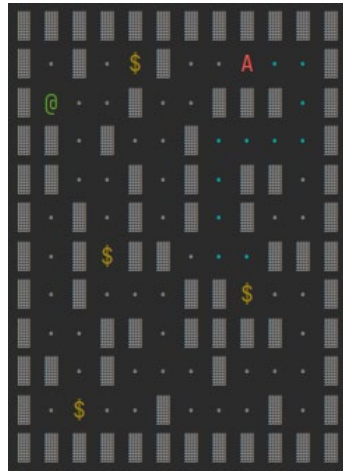


Figure 3: A Maze

The player, ai and treasures all get randomly placed on a path tile in the maze.

Displaying the Maze

Nothing fancy, @ is the player, A is the ai and \$ is treasure. Dots are paths and the brick things are walls. I also add a border around the maze so it looks nicer.

```
private void printMaze()
{
    // print top wall
    for (int x = 0; x < size + 2; x++)
    {
        printWall();
    }

    IO.println();

    for (int y = 0; y < size; y++)
    {
        for (int x = 0; x < size; x++)
        {
            // print left wall
            if (x == 0)
            {
                printWall();
            }
        }
    }
}
```

Treasure Hunt

```
    }

    Tile tile = grid[y][x];

    switch (tile)
    {
        case WALL ->
            printWall();

        case PATH ->
            printPath();

        case PLAYER ->
            printPlayer();

        case AI ->
            printAI();

        case TREASURE ->
            printTreasure();
    }
}

// print right wall
printWall();

IO.println(); // next row
}

// print bottom wall
for (int x = 0; x < size + 2; x++)
{
    printWall();
}

IO.println();
}
```

Figure 4: printMaze

Player Controls

The main game loop is simply getting W, A, S, D inputs from the player and updating the position in the maze. The ai uses the same methods for its movement.

```
while (!maze.isMazeCompleted())
{
    UI.clearScreen();

    maze.displayMazeWithAIPath();

    maze.aiGetMinimaxMove();

    char playerInput;
    Direction moveDirection;
    boolean hasPlayerMoved = false;

    do
    {
        playerInput = UI.getFilteredCharInput('w', 'a', 's',
        'd');

        moveDirection = getDirectionFromInput(playerInput);

        if (maze.canMovePlayer(moveDirection))
        {
            maze.movePlayer(moveDirection);
            hasPlayerMoved = true;
        }
    }
    while (!hasPlayerMoved);

    maze.aiTurn();
}
```

Figure 5: player movement

Internally, it just swaps the player tile around. Thank you IntelliJ auto-complete for the very insightful comments.

Treasure Hunt

```
private void movePlayer(TilePosition newPosition)
{
    if (isTraversableTile(newPosition))
    {
        if (getTileType(newPosition) == Tile.TREASURE)
        {
            playerTreasuresCollected++;
        }

        // Clear old position
        grid[playerPosition.row()][playerPosition.col()] =
Tile.PATH;

        // Update to new position
        playerPosition = newPosition;
        grid[playerPosition.row()][playerPosition.col()] =
Tile.PLAYER;

        checkTreasureStatus();
    }
}
```

Figure 6: movePlayer

Pathfinding

I first attempted to implement A* but that gave me a headache. I instead just implemented a simpler depth first search algorithm. I also ignore game characters for the pathfinding, even if technically if a player or ai is in the way, it would be unreachable.

```
public List<TilePosition> findPath(TilePosition start,
TilePosition goal)
{
    List<TilePosition> path = new ArrayList<>();

    simpleSearch(start, goal, path, new ArrayList<>());

    return path.reversed();
}
```

Treasure Hunt

```
private boolean simpleSearch(TilePosition current, TilePosition
goal, List<TilePosition> path, List<TilePosition> visited)
{
    if (current.equals(goal))
    {
        return true;
    }

    if (!isTraversableTile(current, true) ||
visited.contains(current))
    {
        return false;
    }

    visited.add(current);

    for (TilePosition neighbour :
getTraversableNeighbourTiles(current, true))
    {
        if (simpleSearch(neighbour, goal, path, visited))
        {
            path.add(neighbour);
            return true;
        }

        path.remove(neighbour);
    }

    return false;
}
```

Figure 7: simple search algorithm

The generated path is a list of TilePositions. I created separate maze display methods that overlays a path (or two) on the maze. This looks nice and is also somewhat helpful for debugging.

Greedy

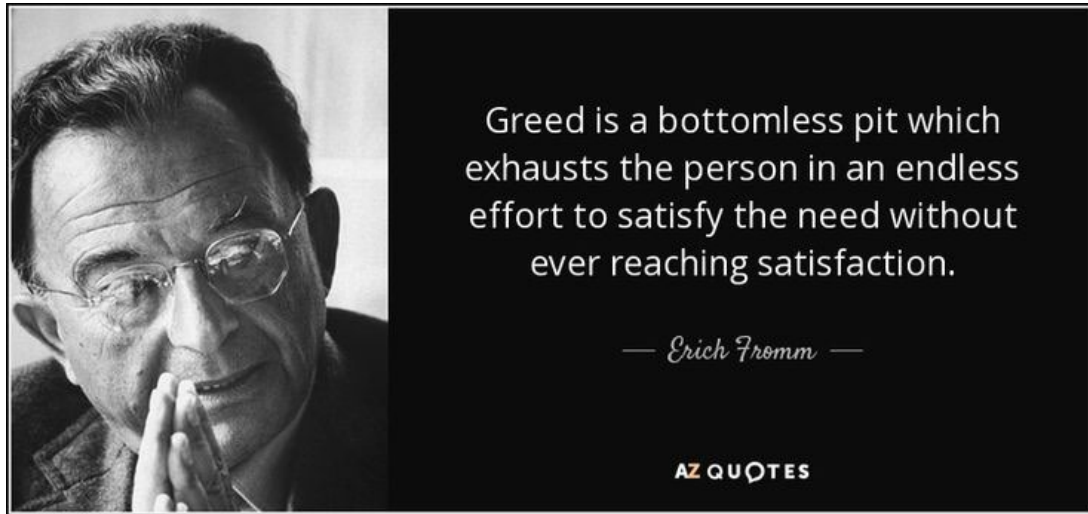


Figure 8: Inspirational Quote

Incredibly simple, it just calculates the route length for each treasure and returns the one with the shortest route.

```
public List<TilePosition> aiGetGreedyMove()
{
    List<TilePosition> treasurePositions =
    getAllTreasurePositions();
    List<List<TilePosition>> pathsToTreasures = new
    ArrayList<>();

    for (TilePosition treasurePosition : treasurePositions)
    {
        List<TilePosition> path = findPath(aiPosition,
    treasurePosition);
        if (!path.isEmpty())
        {
            pathsToTreasures.add(path);
        }
    }

    if (!pathsToTreasures.isEmpty())
    {
        List<TilePosition> shortestPath =
    pathsToTreasures.getFirst();
    }
```

Treasure Hunt

```
        for (List<TilePosition> path : pathsToTreasures)
        {
            if (path.size() < shortestPath.size())
            {
                shortestPath = path;
            }
        }

        return shortestPath;
    }

    return new ArrayList<>();
}
```

Figure 9: Greedy

MiniMax

I have to say I kinda struggled to implement minimax to this project. I at first attempted to minimize the distance to the treasure from the ai and the maximize the player distance. At first, I did it without recursion, but that wasn't really good. Eventually, I managed to make something. (I know the exercise says to use the treasure count as a metric, but I think using minimax to find which treasure to pursue works better for my game, I think?)

It recursively goes over possible ai and player moves, and yeah it works. I made a better evaluation method though. Firstly, if the treasure is near the AI, prioritize that. If the player is close to a treasure, deprioritize it.

Then I calculate the distances to the treasure and multiply it with an urgency metric. That just makes close treasure be more valuable for the AI. Without that it would sometimes decide to go to very far away treasures, which, isn't quite the best strategy.

```
public List<TilePosition> aiGetMinimaxMove()
{
    List<TilePosition> treasurePositions =
    getAllTreasurePositions();
```

Treasure Hunt

```
List<TilePosition> bestPath = new ArrayList<>();
int bestScore = Integer.MIN_VALUE;

for (TilePosition treasure : treasurePositions)
{
    List<TilePosition> aiPath = findPath(aiPosition,
treasure);
    List<TilePosition> playerPath = findPath(playerPosition,
treasure);

    if (aiPath == null || aiPath.isEmpty())
        continue;

    int score = minimax(aiPosition, playerPosition,
treasure, 5, true);

    if (AiMode.DEBUG == aiMode)
    {
        UI.printlnRed("Evaluating Treasure at " + treasure +
": AI Path Length = " + aiPath.size() + ", Player Path Length =
" + (playerPath != null ? playerPath.size() : "N/A") + ", Score
= " + score);
    }

    if (score > bestScore)
    {
        bestScore = score;
        bestPath = aiPath;
    }
}

return bestPath;
}

private int minimax(TilePosition aiPos, TilePosition playerPos,
TilePosition treasure, int depth, boolean maximizingAi)
{
    List<TilePosition> aiPath = findPath(aiPos, treasure);
    List<TilePosition> playerPath = findPath(playerPos,
treasure);
```

Treasure Hunt

```
if (depth == 0 || aiPath == null || playerPath == null)
{
    assert aiPath != null;
    assert playerPath != null;
    return minimaxEval(aiPath.size(), playerPath.size());
}

int bestScore;
if (maximizingAi)
{
    bestScore = Integer.MIN_VALUE;

    for (TilePosition nextAi :
getTraversableNeighbourTiles(aiPos, true))
    {
        int score = minimax(nextAi, playerPos, treasure,
depth - 1, false);

        bestScore = Math.max(bestScore, score);
    }
}
else
{
    bestScore = Integer.MAX_VALUE;

    for (TilePosition nextPlayer :
getTraversableNeighbourTiles(playerPos, true))
    {
        int score = minimax(aiPos, nextPlayer, treasure,
depth - 1, true);

        bestScore = Math.min(bestScore, score);
    }
}

return bestScore;
}

private int minimaxEval(int aiDistance, int playerDistance)
```

Treasure Hunt

```
{
    if (aiDistance <= 1)
    {
        return 1000;
    }

    if (playerDistance <= 1)
    {
        return -1000;
    }

    int diff = playerDistance - aiDistance;

    int urgency = Math.max(1, 15 - Math.min(aiDistance,
playerDistance));

    return diff * urgency;
}
```

Figure 10: MiniMax

Example Game State

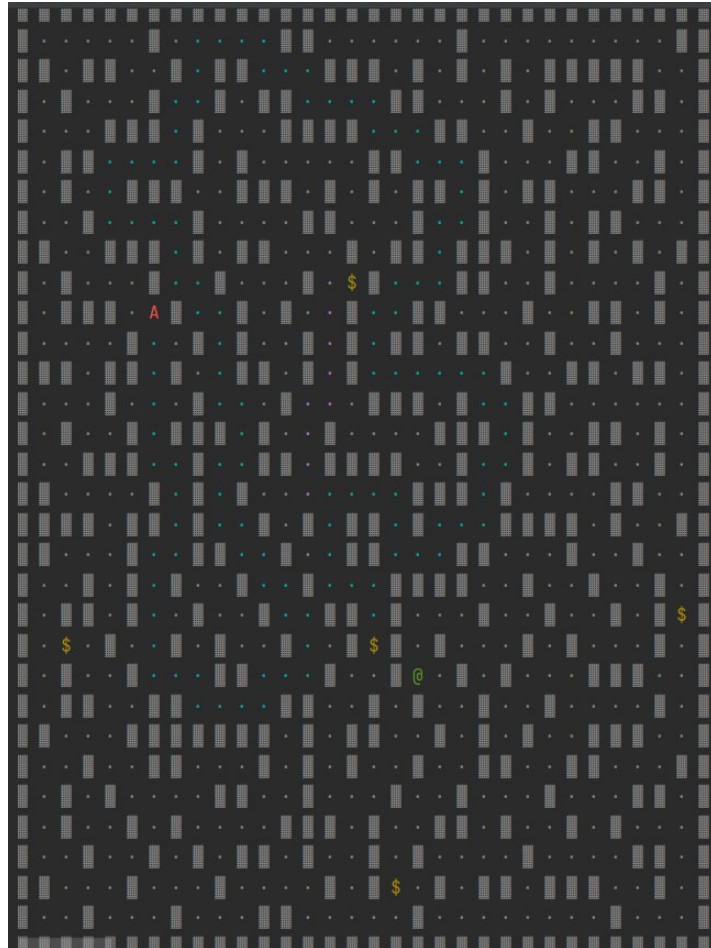


Figure 11: another maze! big!

Score - You: 0 | AI: 0

AI Greedy Path: ...big array

AI MiniMax Path: ...big array

Greedy Treasure Positions: TilePosition[row=20, col=15]

Minimax Treasure Positions: TilePosition[row=8, col=14]

Evaluating Treasure at TilePosition[row=8, col=14]: AI Path
Length = 118, Player Path Length = 44, Score = -73

Evaluating Treasure at TilePosition[row=19, col=29]: AI Path
Length = 188, Player Path Length = 46, Score = -141

Evaluating Treasure at TilePosition[row=20, col=1]: AI Path
Length = 159, Player Path Length = 47, Score = -111

Treasure Hunt

```
Evaluating Treasure at TilePosition[row=20, col=15]: AI Path  
Length = 115, Player Path Length = 27, Score = -87  
Evaluating Treasure at TilePosition[row=28, col=16]: AI Path  
Length = 130, Player Path Length = 12, Score = -585
```

Figure 12: Debug Output

There is a special ai mode that displays debug information, like the paths and everything. I primarily used this for testing. It can be disabled in the game settings.